

Análise Comparativa de Desempenho de Algoritmos de Ordenação em Diferentes Cenários de Entrada

Felipe Rochoel Costa

Graduação em Engenharia de Computação
Universidade Tecnológica Federal do Paraná (UTFPR) – Campus Apucarana
Apucarana – PR – Brasil
`feliperochoelcosta@alunos.utfpr.edu.br`

Abstract. This paper presents an empirical and comparative performance analysis of three widely studied sorting algorithms: Bubble Sort, Merge Sort, and Quick Sort. The evaluation focuses on execution time and the number of key comparisons and element swaps across three initial data ordering scenarios (random, inversely sorted, and nearly sorted lists) with dataset sizes ranging from 10,000 to 1,000,000 elements. The experimental results validate the theoretical asymptotic complexities, highlight the overhead of quadratic algorithms in large volumes of data, and explore the practical efficiency trade-offs between divide-and-conquer approaches. Source code and experimental datasets are publicly available at <https://github.com/rochoel-felipe/analise-algoritmos-ordenacao>.

Resumo. Este artigo apresenta uma análise empírica e comparativa de desempenho de três algoritmos de ordenação amplamente estudados: Bubble Sort, Merge Sort e Quick Sort. A avaliação centra-se no tempo de execução e no número de comparações de chaves e trocas de elementos sob três cenários de ordenação inicial dos dados (listas aleatórias, inversamente ordenadas e quase ordenadas), com tamanhos de conjuntos de dados variando de 10.000 a 1.000.000 de elementos. Os resultados experimentais validam as complexidades assintóticas teóricas, evidenciam o gargalo de algoritmos quadráticos em grandes volumes de dados e exploram as compensações de eficiência prática entre as abordagens de divisão e conquista. O código-fonte e os dados experimentais estão disponíveis publicamente em <https://github.com/rochoel-felipe/analise-algoritmos-ordenacao>.

1 Introdução

A ordenação de dados constitui um dos problemas mais fundamentais e exaustivamente explorados no âmbito da Ciência da Computação e da Engenharia de Software. Definida

como o processo de arranjar uma coleção de registros em uma sequência linear determinada por uma relação de ordem total, a ordenação eficiente é um requisito crítico para a otimização de operações subsequentes, tais como buscas binárias, eliminação de duplicatas e processamento de bancos de dados.

Com o advento do *Big Data* e o crescimento exponencial do volume de informações manipuladas por sistemas computacionais modernos, a escolha do algoritmo de ordenação ideal deixou de ser uma mera decisão conceitual para tornar-se um fator determinante no consumo de recursos, na latência e na escalabilidade de softwares. Diferentes algoritmos apresentam comportamentos distintos que dependem diretamente não apenas do número de elementos a ordenar, mas também do estado de ordenação prévio desse conjunto de dados.

Este trabalho tem como objetivo central realizar um estudo comparativo e experimental de três algoritmos de ordenação proeminentes: Bubble Sort, Merge Sort e Quick Sort. A análise avalia o impacto do tamanho da entrada (vetores de $n \in \{10.000; 100.000; 1.000.000\}$ elementos) e o comportamento sob diferentes distribuições iniciais dos dados (listas aleatórias, inversamente ordenadas e quase ordenadas). Por meio da mensuração do tempo computacional de execução e da contagem de operações primitivas (comparações e trocas), busca-se confrontar os achados práticos com as previsões matemáticas da análise assintótica clássica.

2 Fundamentação Teórica

A eficiência de um algoritmo é formalmente avaliada por meio da notação assintótica grande- O , que descreve o limite superior do crescimento do tempo de execução em função do tamanho da entrada n . Os algoritmos selecionados pertencem a duas classes principais de complexidade e estratégias de projeto distintas [1, 3].

2.1 O Algoritmo Bubble Sort e sua Complexidade

O Bubble Sort é um algoritmo elementar iterativo e baseado em comparações. O princípio operacional consiste em percorrer o vetor repetidamente a partir do início, comparando pares de elementos adjacentes e realizando uma operação de troca (*swap*) sempre que dois elementos consecutivos estiverem fora da ordem desejada. Esse procedimento faz com que os maiores elementos “flutuem” gradualmente para as posições finais.

Do ponto de vista analítico, o Bubble Sort possui complexidade de tempo de pior caso e caso médio em $O(n^2)$. O pior caso ocorre quando o vetor está em ordem estritamente inversa, exigindo $n(n-1)/2$ comparações e o mesmo número de trocas. Com a otimização de uma variável sinalizadora (*flag*) que interrompe a execução caso nenhuma troca ocorra em uma passagem completa, o algoritmo atinge $O(n)$ no melhor caso — quando o vetor já está ordenado. Sua complexidade de espaço é $O(1)$, classificando-o como algoritmo *in-place*. Apesar de didaticamente difundido e simples de implementar, sua eficiência degrada de maneira proibitiva em conjuntos de dados massivos.

2.2 O Algoritmo Merge Sort e sua Complexidade

O Merge Sort é um algoritmo de ordenação estável fundamentado no paradigma de “dividir para conquistar”, proposto por John von Neumann em 1945 [3]. O funcionamento divide-se em três etapas recursivas:

1. **Divisão:** O vetor original é segmentado em dois subvetores de tamanho aproximadamente igual, $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$.
2. **Conquista:** Cada subvetor é ordenado recursivamente por novas chamadas ao Merge Sort. Quando um subvetor atinge tamanho unitário, está trivialmente ordenado.
3. **Combinação (*merge*):** Os subvetores ordenados são intercalados de forma linear para recompor um único vetor totalmente ordenado.

A propriedade mais notável do Merge Sort é sua consistência temporal: garante $O(n \log n)$ para melhor, pior e caso médio, pois a árvore de recorrência mantém profundidade $\log n$ e o custo de intercalação em cada nível é $O(n)$. No entanto, a operação de intercalação convencional não é *in-place*, requerendo um vetor auxiliar de tamanho $O(n)$, o que pode limitar sua viabilidade em ambientes com restrição severa de memória.

2.3 O Algoritmo Quick Sort e sua Complexidade

O Quick Sort, idealizado por C. A. R. Hoare em 1959 [3], emprega o paradigma de divisão e conquista em torno de um elemento de referência denominado *pivô*. A operação crítica é o particionamento, que rearranja os elementos de modo que todos os itens menores que o pivô fiquem à sua esquerda e os maiores ou iguais, à sua direita. Após o particionamento, o pivô ocupa sua posição definitiva e o processo é aplicado recursivamente nos dois subvetores resultantes.

Em cenários médios e favoráveis, o Quick Sort exibe complexidade $O(n \log n)$ e, devido à constante multiplicativa baixa e à excelente localidade de referência para caches de CPU, costuma superar o Merge Sort na prática. Contudo, sua eficiência é altamente sensível à escolha do pivô: quando o pivô é sistematicamente o menor ou o maior elemento do conjunto — situação comum em entradas já ordenadas ou inversamente ordenadas com pivô fixo no primeiro elemento —, a árvore de chamadas recursivas atinge profundidade n , degradando a complexidade para $O(n^2)$. O consumo de espaço na pilha de execução varia de $O(\log n)$ no caso médio a $O(n)$ no pior cenário.

3 Método e Procedimentos

3.1 Ambiente de Testes

Os algoritmos foram implementados em linguagem C e compilados com o GCC (MinGW). Os experimentos foram executados de forma isolada na seguinte configuração de hardware e software:

- **Processador:** AMD Ryzen 5 5600G (3,90 GHz)
- **Memória RAM:** 16 GB DDR4 (com pilha de 64 MB alocada dinamicamente para o Quick Sort)
- **Sistema Operacional:** Windows 11 Pro (64 bits, versão 24H2)
- **Compilador:** GCC (MinGW) via Visual Studio Code

A medição do tempo de execução foi realizada com temporizadores de alta precisão da biblioteca `<time.h>`, capturando o intervalo exato entre o início e o término da rotina de ordenação. Para quantificar o esforço computacional de forma independente do hardware, contadores internos foram inseridos no código-fonte para registrar o número exato de comparações de chaves realizadas e de trocas de posição executadas.

O código-fonte, os roteiros de testes e os conjuntos de dados brutos estão disponibilizados publicamente no repositório: <https://github.com/rochoel-felipe/analise-algoritmos-ordenacao>.

3.2 Cenários Avaliados

Cada algoritmo foi submetido a testes combinando três tamanhos de vetores ($n = 10.000$, $n = 100.000$ e $n = 1.000.000$) em três perfis de ordenação inicial. Uma restrição laboratorial foi aplicada ao Bubble Sort, limitando seu teste máximo a $n = 500.000$ para evitar sobrecarga temporal inviável; para $n = 1.000.000$, o tempo esperado foi extrapolado a partir dos dados coletados. Os perfis definidos foram:

- **Listas Aleatórias:** Elementos gerados pseudoaleatoriamente com semente fixa (`srand(42)`), garantindo reprodutibilidade entre as rodadas.
- **Listas Inversamente Ordenadas:** Vetores preenchidos em ordem estritamente decrescente — pior caso teórico tanto para o Bubble Sort quanto para o Quick Sort com pivô fixo.
- **Listas Quase Ordenadas:** Vetores originalmente ordenados de forma crescente nos quais aproximadamente 2% dos elementos tiveram suas posições deliberadamente permutadas.

Cada configuração foi executada em três rodadas independentes. As métricas de comparações e trocas são determinísticas (idênticas entre rodadas); os tempos de processamento reportados representam a média aritmética das três execuções.

4 Resultados e Discussão

4.1 Desempenho em Listas Aleatórias

A Tabela 1 apresenta o comportamento dos algoritmos sobre dados gerados arbitrariamente.

Tabela 1: Desempenho dos algoritmos de ordenação em listas aleatórias.

Algoritmo	Tamanho (n)	Tempo Médio (ms)	Comparações	Trocas
Bubble Sort	10.000	126,67	49.994.535	24.845.189
	100.000	21.494,33	499.844.889	250.489.970
	500.000*	544.215,00	124.999.422.355	62.425.097.729
Merge Sort	10.000	2,67	120.439	133.616
	100.000	21,33	1.536.713	1.668.928
	1.000.000	224,33	18.674.332	19.951.424
Quick Sort	10.000	<1	166.685	78.147
	100.000	8,33	2.090.695	1.001.984
	1.000.000	100,00	35.550.416	10.268.671

*Limitado a 500.000 elementos em razão da complexidade quadrática do algoritmo.

No cenário aleatório, os resultados práticos alinham-se com os limites teóricos da análise assintótica. O crescimento do tempo e das operações do Bubble Sort evidencia o custo quadrático $O(n^2)$: ao escalar de 10.000 para 100.000 elementos (fator 10), o tempo médio passou de 126,67 ms para 21.494,33 ms (aumento de aproximadamente 170 vezes, próximo da escala 10^2). Para $n = 500.000$, o tempo ultrapassou 9 minutos.

Os algoritmos baseados em divisão e conquista comprovaram sua eficiência superior. O Merge Sort manteve-se linearitmicamente estável, ordenando 1.000.000 de registros em 224,33 ms. O Quick Sort consolidou-se como o mais veloz neste cenário: com tempo inferior a 1 ms para $n = 10.000$ e apenas 100,00 ms para o vetor máximo. Essa vantagem sobre o Merge Sort decorre da menor constante multiplicativa e da ausência de alocação de memória auxiliar.

4.2 Desempenho em Listas Inversamente Ordenadas

Os dados para o cenário de ordenação inversa constam na Tabela 2.

O cenário inversamente ordenado revelou comportamentos de extrema relevância teórica. A escolha estática do primeiro elemento como pivô forçou o Quick Sort a incorrer sistematicamente em particionamentos degenerados e completamente desbalanceados (subvetores de tamanhos 0 e $n - 1$), degradando sua complexidade para o pior caso $O(n^2)$. Para $n = 1.000.000$, o algoritmo realizou quase 500 bilhões de comparações, demandando 1.000.239,67 ms — aproximadamente 16,7 minutos — de processamento.

O Bubble Sort também atingiu seu ápice de ineficiência: cada comparação resultou em uma troca, gerando a quantidade máxima de operações $n(n - 1)/2$. O Merge Sort, por outro lado, demonstrou resiliência assintótica completa: indiferente à disposição desfavorável dos dados, manteve 158,33 ms para o vetor máximo, consagrando-se como a abordagem mais segura e robusta diante de distribuições inversas.

Tabela 2: Desempenho dos algoritmos em listas ordenadas de forma inversa.

Algoritmo	Tamanho (n)	Tempo Médio (ms)	Comparações	Trocas
Bubble Sort	10.000	152,00	49.995.000	49.995.000
	100.000	18.448,33	4.999.950.000	4.999.950.000
	500.000*	459.820,67	124.999.750.000	124.999.750.000
Merge Sort	10.000	<1	64.608	133.616
	100.000	19,67	815.024	1.668.928
	1.000.000	158,33	9.884.992	19.951.424
Quick Sort	10.000	106,33	49.995.000	25.009.999
	100.000	10.002,00	4.999.950.000	2.500.099.999
	1.000.000	1.000.239,67	499.999.500.000	250.000.999.999

*Limitado a 500.000 elementos em razão da complexidade quadrática do algoritmo.

4.3 Desempenho em Listas Quase Ordenadas

Os dados para vetores quase totalmente ordenados constam na Tabela 3.

Tabela 3: Desempenho dos algoritmos em listas quase ordenadas.

Algoritmo	Tamanho (n)	Tempo Médio (ms)	Comparações	Trocas
Bubble Sort	10.000	76,33	49.806.809	1.286.410
	100.000	4.019,33	2.723.276.019	39.529.560
	500.000*	22.727,67	15.617.581.497	144.816.562
Merge Sort	10.000	2,00	105.886	133.616
	100.000	19,33	1.017.935	1.668.928
	1.000.000	165,00	10.263.537	19.951.424
Quick Sort	10.000	1,67	2.021.659	149.163
	100.000	2.305,00	226.830.634	537.932
	1.000.000	477.187,33	467.783.498.400	1.280.759

*Limitado a 500.000 elementos em razão da complexidade quadrática do algoritmo.

O cenário quase ordenado evidenciou dois comportamentos de grande interesse prático. O Bubble Sort beneficiou-se expressivamente da pré-ordenação: com apenas 144.816.562 trocas para $n = 500.000$ (contra 124.999.750.000 no cenário inverso), e um tempo médio de 22.727,67 ms, a redução é drástica em relação ao pior caso. A otimização por *flag* permite ao algoritmo encerrar cada passagem mais cedo, diminuindo o número efetivo de iterações. Ainda assim, o volume de comparações permaneceu elevado (cerca de 15 bilhões para $n = 500.000$), pois o algoritmo precisa percorrer o vetor até confirmar a ausência total de inversões.

O Quick Sort com pivô fixo no primeiro elemento também sofreu penalização

neste cenário, embora de forma menos severa que no caso inverso. Como os primeiros elementos já se encontravam próximos de suas posições definitivas na sequência crescente, as divisões geradas foram desbalanceadas, resultando em 477.187,33 ms para $n = 10^6$ e em curvas de crescimento próximas a $O(n^2)$. Vale notar, contudo, que o número de trocas foi extremamente baixo (apenas 1.280.759 para $n = 1.000.000$), confirmando que o custo recai quase inteiramente sobre o particionamento, não sobre os rearranjos de elementos. Tal comportamento reitera a necessidade prática de estratégias como pivô aleatório ou mediana de três em implementações reais.

O Merge Sort reiterou sua estabilidade insensível ao arranjo inicial, resolvendo o vetor máximo em 165,00 ms — o menor tempo registrado entre todos os algoritmos neste cenário.

5 Conclusão

Este estudo confrontou as formulações da teoria assintótica com o comportamento empírico observado nos experimentos. O Bubble Sort provou-se inadequado para conjuntos extensos — o ônus do crescimento quadrático $O(n^2)$ é intransponível —, exibindo melhora expressiva apenas em estruturas quase ordenadas, onde a otimização por *flag* reduz sensivelmente o número de trocas e, conseqüentemente, o tempo total.

As técnicas de divisão e conquista reafirmaram sua supremacia. O Merge Sort destacou-se pela previsibilidade temporal absoluta, mantendo $O(n \log n)$ em todos os cenários ao custo de $O(n)$ de espaço auxiliar. O Quick Sort consolidou-se como a ferramenta mais veloz no caso médio (dados aleatórios), mas sua dependência crítica da estratégia de escolha do pivô foi empiricamente exposta nos cenários inverso e quase ordenado, onde regrediu ao desempenho quadrático. Notavelmente, neste último cenário, o Quick Sort realizou um número mínimo de trocas, evidenciando que o gargalo reside exclusivamente na fase de particionamento.

Conclui-se que a seleção do algoritmo ideal deve equilibrar não apenas o volume de dados, mas o conhecimento prévio sobre a distribuição estrutural da entrada. Para cenários de uso geral, o Merge Sort oferece maior segurança e previsibilidade; para dados tipicamente aleatórios onde desempenho médio é prioritário, o Quick Sort com escolha de pivô robusta (e.g., mediana de três ou pivô aleatório) apresenta vantagens práticas relevantes.

Referências

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2012). *Algoritmos: Teoria e Prática*. Elsevier, Rio de Janeiro, 3ª edição.
- [2] Góes, W. (2026). *Material Didático da Disciplina de Estruturas de Dados 2: Aulas de Bubble Sort, Merge Sort e Quick Sort*. UTFPR, Campus Apucarana.

- [3] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2nd edition.